Enterprise Portal offers your customers one central place to view install and upgrade instructions for each version of your software. They can set up their environment, and get troubleshooting support by uploading support bundles.

By connecting your own GitHub repo, you can serve versioned documentation to support your releases, auto-generate Helm chart reference docs, and distribute Terraform modules through a license-gated proxy registry.

Content is driven by entitlements and channel assignment, so each customer sees only what applies to them.

Note: Features described here are in alpha and may not yet appear in the public Replicated docs.

---

1. Connect Your Git Repo

Enterprise Portal content is driven by a GitHub repo you control. Connecting a repo is a prerequisite to offer versioned docs, advanced theming, and Terraform module delivery.

1. (Optional) Fork our [example repo]
2. Go to **Content** tab in Vendor Portal
3. Click **Connect to GitHub** and authorize the Replicated GitHub App
4. Select your repo

Content syncs automatically on push via GitHub webhook.

You can also trigger a manual sync from the **Content** tab.

Via API:

```
None
# Your APP_ID is visible in the Vendor Portal URL when viewing your app,
# or via:
```

```
replicated api get /v3/apps | jq '.apps[] | {id, name}'

# List your content repos to get the REPO_ID:
replicated api get /v3/app/<APP_ID>/enterprise-portal/content-repos

# Trigger a sync:
replicated api post
/v3/app/<APP_ID>/enterprise-portal/content-repos/<REPO_ID>/sync
```

---

2. Customize Content

Table of Contents

The `toc.yaml` file at the root of your repo defines the sidebar navigation:

```
None
navigation:
  - title: Getting Started
    icon: rocket
    page: pages/getting-started.md

  - title: Installation
    icon: download
    items:
      - title: Requirements
        page: pages/installation/requirements.md
      - title: Helm Installation
        page: pages/installation/helm.md
        visible_when:
          entitlements:
            - isHelmInstallEnabled
      - title: Air Gap Installation
        page: pages/installation/airgap.md
…
```

Each navigation item has a `title` and one of the following content types:

Key	What it does
`page`	Renders a markdown file from your repo (e.g. `pages/getting-started.md`)
`terraform_module`	Generates docs from a Terraform module source URI (See Terraform Modules below)
`helm_chart`	Generates reference docs from a Helm chart in your promoted release (See Helm Reference Docs below)
`items`	Nests child navigation items to create expandable sections

Every item also supports `icon` and `visible_when` (see Visibility below).

Supported icon values: `rocket`, `download`, `cloud`, `settings`, `wrench`, `life-buoy`, `file-text`, `star`, `book`, `shield`, `package`, `refresh-cw`, `database`, `key`. If omitted, defaults to `book`.

Complete Example

Here's a `toc.yaml` showing all content types together:

```
None
navigation:
  - title: Getting Started
    icon: rocket
    page: pages/getting-started.md

  - title: Installation
    icon: download
    items:
```

```
      - title: Requirements
        page: pages/installation/requirements.md
      - title: Helm Installation
        page: pages/installation/helm.md
        visible_when:
          entitlements:
            - isHelmInstallEnabled
      - title: Air Gap Installation
        page: pages/installation/airgap.md
        visible_when:
          entitlements:
            - isAirgapSupported

  - title: Infrastructure
    icon: database
    items:
      - title: AWS Module
        terraform_module:
github.com/your-org/your-terraform//modules/aws?ref=v1.0.0
        visible_when:
          entitlements:
            - isAWSEnabled

  - title: Reference
    icon: book
    items:
      - title: My App Chart
        helm_chart:
          name: my-app
overrides:
  home: pages/getting-started.md
```

Template Variables

Enterprise Portal content supports Mustache-style templating with `{{ }}` syntax for dynamic rendering.

Available Variables

Variable	Description
`{{ app.name }}`	Your application name
`{{ app.slug }}`	URL-friendly app identifier
`{{ customer.name }}`	Customer's name
`{{ customer.email }}`	Customer's email
`{{ customer.id }}`	Customer ID
`{{ channel.name }}`	Release channel (Stable, Beta, etc.)
`{{ channel.slug }}`	URL-friendly channel identifier
`{{ release.version }}`	Current release version
`{{ license.* }}`	Any field from the license object (e.g. `{{ license.licenseId }}`, `{{ license.expiresAt }}`)
`{{ entitlements.* }}`	Any entitlement value — built-in or custom (see below)

Note: `{{ release.version }}` depends on the customer having reported an active instance. It may be empty for new customers who haven't installed yet. Consider using `{{ channel.name }}` as a more reliable alternative for version-dependent content.

Conditionals

`{{#if}}` — Show content when a value is truthy. Supports nesting.

```
None
{{#if entitlements.isHAEnabled}}
## High Availability Setup
For production deployments, enable HA mode...
{{/if}}
```

Values are considered falsy if they are null, empty string, `false`, `"false"`, `"0"`, or `0`. Everything else is truthy.

`{{#ifEquals}}` — Show content when a value equals a specific string. Useful for per-customer or per-channel content.

```
None
{{#ifEquals channel.name "Stable"}}
You are on the Stable channel. Updates are less frequent but thoroughly
tested.
{{/ifEquals}}

{{#ifEquals customer.name "Acme Corp"}}
## Acme-Specific Configuration
Your dedicated endpoint is acme.example.com
{{/ifEquals}}
```

The comparison value must be quoted (double or single quotes). Supports nesting.

Entitlements

Entitlements are accessible as `{{ entitlements.<name> }}` for interpolation and `{{#if entitlements.<name>}}` for conditionals. There are two kinds:

- **Built-in flags** — set on the license via Vendor Portal: `isHelmInstallEnabled`, `isAirgapSupported`, `isEmbeddedClusterDownloadEnabled`, `isEmbeddedClusterMultiNodeEnabled`, `isKotsInstallEnabled`, `isHelmAirgapEnabled`
- **Custom Entitlements** — any that you define (e.g. `isHAEnabled`, `isEnterpriseEnabled`)

Example:

```
None
{{#if entitlements.isHelmInstallEnabled}}
```

```
## Helm Installation
Follow these steps to install using Helm...
{{/if}}

{{#if entitlements.isHAEnabled}}
## High Availability Setup
For production deployments, enable HA mode...
{{/if}}
```

Visibility

Use `visible_when` on any navigation item to control which customers see which content. All conditions must be satisfied (AND logic).

```
None
- title: Enterprise Features
  page: pages/enterprise.md
  visible_when:
    entitlements:
      - isEnterpriseEnabled
    channel:
      include: [Stable, Beta]
      exclude: [Internal]
    customer_type:
      include: [paid]
```

Note: `entitlements` filtering is fully implemented. `channel` include/exclude and `customer_type` include are parsed and stored but **not yet evaluated in the frontend** — these will be enforced in a future release.

Custom Theme

Include a `theme.yaml` file in your repo to override branding for any version of your docs.

Supported fields:

Field	Type	Notes
`title`	string	Portal title override, max 255 chars
`overview`	string	Portal description, max 2000 chars
`logo`	file path	Logo image, max 2MB (resolved to data URI)
`favicon`	file path	Favicon, max 500KB
`primaryColor`	hex	Primary brand color (`#RGB` or `#RRGGBB`)
`secondaryColor`	hex	Secondary brand color
`linkColor`	hex	Link color
`linkHoverColor`	hex	Link hover color
`background`	enum	`minimal`, `custom`, or `image`
`backgroundImage`	file path	Background image, max 5MB (requires `background: image`)
`customColor1` / `customColor2`	hex	Custom gradient colors (required when `background: custom`)
`headerColor` / `headerGradientEnd`	hex	Header gradient
`sidebarColor` / `sidebarGradientEnd`	hex	Sidebar gradient

Field	Type	Notes
`contentBackgroundColor` / `contentBackgroundGradientEnd`	hex	Content area gradient
`customCSS`	inline string	Inline CSS overrides, max 50KB
`customCSSFile`	file path	Path to external CSS file, max 50KB
`contact`	string	Support contact info, max 255 chars
`supportPortalLink`	URL	External support link, max 2048 chars (http/https only)

Downloadable Assets

You can distribute files (scripts, checklists, configuration templates, etc.) through Enterprise Portal by committing them to an `assets/` directory in your content repo.

Repo Structure

```
your-content-repo/
├── assets/
│   ├── deploy.sh
│   └── onboarding-checklist.pdf
├── pages/
│   └── resources.md
└── toc.yaml
```

Any files inside the `assets/` directory are synced automatically alongside pages. Each file can be up to 50MB. The asset count appears in the Content tab sync status.

Linking to Assets

Use the `{{asset "path"}}` template tag in your markdown to generate a download link:

```
None
## Resources
- [Deployment Script]({{asset "assets/deploy.sh"}})
- [Onboarding Checklist]({{asset "assets/onboarding-checklist.pdf"}})
```

The tag expands to a URL that authenticates the customer via their session and serves the file directly. Assets are version-aware, so each content branch can have its own set of files.

The path inside the tag is relative to the repo root and must start with `assets/`.

---

3. Preview Enterprise Portal

Before inviting any real customers, use Preview to see exactly what they'll experience.

Vendor Portal > **Content** > **Preview** section.

Select a content branch and a (test) customer, then click **Open Preview**.

This opens an embedded view of the full Enterprise Portal experience as that customer would see it, with their entitlements, channel, and license data applied.

You can switch branches and customers from the preview toolbar without leaving the page.

(You can also open any live customer's portal - [View a Customer's Enterprise Portal](#).)

---

4. Helm Reference Docs

Use this section if your releases contain Helm charts. Reference docs are generated automatically when you promote a release — no manual authoring required.

Any Helm charts in a promoted release will generate reference docs. Add a `helm_chart` entry to `toc.yaml`:

```
None
- title: Reference
  icon: book
  items:
    - title: My App Chart
      helm_chart:
        name: my-app
    - title: Operator Chart
      helm_chart:
        name: my-app-operator
```

The `name` must match a chart name in your promoted release (the Helm chart name from `Chart.yaml`, not the release name).

Process

When you promote a release to a channel, Enterprise Portal automatically:

1. Extracts `values.yaml` and `Chart.yaml` from the release's chart archives
2. Parses all configurable values with types, defaults, and inline comments
3. Groups values into sections (Container Images, Service, Ingress, Resources, Persistence, Authentication, etc.)
4. AI improves descriptions for any values lacking comments
5. Generates deployment scenarios with example YAML

Note: Steps 4 and 5 run asynchronously. You may see "Generating..." placeholders for a few minutes after promotion. Refresh the page — AI-generated content appears once processing completes.

A content hash ensures regeneration only runs when chart content actually changes.

Content

Generated reference page shows:

- **Chart header** — name, version, description
- **Notes** — warnings, security defaults, and important configuration callouts
- **Values table** — for each section: key path, type, default, description, valid options, and security notes
- **Deployment scenarios** — example `values.yaml` snippets for common configurations (e.g., production, dev, HA)
- **Version comparison** — select two versions to see a diff of values that were added, removed, or changed between releases

You can control comments and visibility in your `values.yaml`:

- Inline comments (`# ...`) become value descriptions
- Add `# @docs:skip` or `# @docs:internal` to exclude values from the reference
- Fields with names like `password`, `secret`, `token`, or `apiKey` are automatically flagged with security notes

---

5. Versioned Docs

Use this section if you need to maintain different documentation for different versions of your software — for example, distinct install steps or caveats for specific releases.

Each Git branch in your content repo (other than `main`) becomes a selectable version in the customer-facing portal.

Each version has its own table of contents, markdown pages, and theme/branding overrides.

Process

- **Initial sync**: When you connect a repo, all branches are discovered and synced.

- **Push sync**: When you push to any branch, a GitHub webhook triggers a sync for that branch only. Pushing a new branch for the first time automatically creates a new version.
- **`main` is synced but hidden**: The `main` branch content is stored in the database but never shown to customers. It's never selectable in the dropdown and never used as a default. You can use `main` as a draft/working branch.

Version Dropdown

Customers viewing Enterprise Portal see a version dropdown in the header when there are two or more versions available on their channel.

Versions are sorted by **reverse lexicographic order** on the branch name. The highest-sorting branch name is first in the list and becomes the default.

When a customer visits the portal:

1. If they've previously selected a version (stored in a browser cookie), that version is used
2. Otherwise, the **first version** in the sorted list (highest branch name) is the default
3. If no versions exist, built-in pages (install, support, etc.) still render without a version selector

Naming Your Branches

Recommended: Name your branches to match your release `version_label` values (e.g., branch `1.0.0` for release version `1.0.0`). This enables version gating by channel — customers only see doc versions for releases on their channel. A `v` prefix mismatch is handled automatically (branch `v1.0.0` matches label `1.0.0` and vice versa).

Your naming convention also controls the dropdown order — versions are sorted in reverse lexicographic order, so the highest branch name becomes the default:

Examples:

- `1.0.0`, `2.0.0`, `2.1.0` → default = `2.1.0`
- `v1.0`, `v2.0`, `v2.1` → default = `v2.1`

Note: If your branch names don't match any version labels (e.g., `release-1`, `release-2`), version gating is skipped and all branches are shown to all customers.

Adding Versions

1. Create a new branch matching your release version label: `git checkout -b 2.0.0`
2. Update the docs content for the new version
3. Push the branch — Enterprise Portal syncs automatically and the new version appears in the dropdown for customers on a channel with that release

The system does not create branches for you — you must manually create branches that correspond to your releases.

Version Gating by Channel

The version dropdown is filtered by the customer's assigned channel. Customers only see branches whose names match a `version_label` on a release promoted to their channel. For example, if a customer is on the Stable channel with releases `1.0.0` and `2.0.0`, they will only see branches named `1.0.0` and `2.0.0` (or `v1.0.0` / `v2.0.0` — the `v` prefix is handled automatically).

If no branches match any version labels (e.g., you use a naming convention like `release-1` that doesn't correspond to version labels), the filter is skipped and all branches are shown.

To use version gating: name your content repo branches to match your release `version_label` values. The Vendor Portal preview also respects this filtering — when you select a customer in the preview, the Content Branch dropdown shows only the branches that customer would see.

Version URL

Versioned pages use the pattern `/{version}/path/to/page`:

- `/2.0.0/installation/helm`
- `/1.0.0/getting-started`

When a customer switches versions via the dropdown, the portal swaps the version segment in the URL and navigates to the same page path in the new version.

Removing Versions

To unpublish a version from the dropdown, delete the branch. The version disappears after the next sync.

Overrides

`main` content is never shown to customers. In `toc.yaml`, the docs template currently supports an `overrides` map.

`overrides` — Currently the only supported override is `home`, which replaces the default version landing page with a custom markdown page:

```
None
overrides:
  home: pages/getting-started.md
```

---

6. Terraform Modules

Use this section if you distribute Terraform modules and want to gate access via customer licenses. Enterprise Portal provides two complementary capabilities:

- **Module Reference Docs** — Generated from `terraform_module` entries in your content repo's `toc.yaml`. These display inputs, outputs, requirements, and AI-generated usage examples on the portal.
- **Proxy Registry** — Lets customers pull modules directly via `terraform init` using their license ID. Requires linking a separate Terraform repo and promoting releases with matching version labels.

Module Reference Docs

Reference Terraform modules in your content repo's `toc.yaml`:

```
None
- title: Infrastructure
  icon: database
  items:
    - title: AWS
      terraform_module: github.com/your-org/your-terraform//aws?ref=main
      visible_when:
        entitlements:
          - isAWSEnabled
    - title: Azure
      terraform_module: github.com/your-org/your-terraform//azure?ref=main
      visible_when:
        entitlements:
          - isAzureEnabled
    - title: GCP
      terraform_module: github.com/your-org/your-terraform//gcp?ref=main
      visible_when:
        entitlements:
          - isGCPEnabled
```

- Uses standard Terraform module source syntax (double slash `//` for subdirectories)
- `ref` specifies the Git ref (tag, branch, or commit)
- Control visibility per customer using `visible_when` entitlements
- The GitHub App installation used for your content repo must also have access to the Terraform module repo

For each module, Enterprise Portal parses `variables.tf`, `outputs.tf`, `versions.tf`, and `README.md`, then generates reference documentation for requirements, inputs, outputs, and AI-generated usage examples.

Note: AI-generated usage examples and descriptions are produced asynchronously. You may see "Generating..." placeholders for a few minutes after sync. Refresh the page — content appears once processing completes.

Note: Terraform module features may require enablement on your app. If terraform content isn't appearing after proper configuration, contact Replicated support.

Proxy Registry

Enterprise Portal acts as a **Terraform module proxy registry**, allowing customers to pull modules directly via `terraform init` using their existing license ID.

Vendor Setup (one-time)

Step 1: Install Replicated GitHub App

If you've already connected a content repo, your GitHub App installation may already cover the Terraform repo. Confirm that the Terraform repo is included in the app's repo access list. If not, go to **Enterprise Portal > Content** and click **Install GitHub App** to add or update access.

Step 2: Link the Terraform repo

In the **Content** tab, use the **Terraform Repository** card to link your repo:

1. Click **Link Terraform Repository**
2. Select your GitHub org and the repo containing your Terraform modules
3. Save

This is a **separate** repo from your content repo. Unlike the content repo, Enterprise Portal doesn't clone or parse the Terraform repo. The link is only used to generate short-lived GitHub tokens at runtime when customers fetch modules via the proxy registry.

Note: Module **reference documentation** on the portal is driven by `terraform_module` URIs in your content repo's `toc.yaml` (see above), not by this linked repo.

Step 3: Promote releases

Create releases and promote them to channels. The `version_label` on each channel release (e.g. `1.0.0`) becomes the Terraform module version available to customers on that channel.

Important: Version labels must match git tags with a `v` prefix. For example, a release with `version_label: 1.0.0` requires git tag `v1.0.0` in the Terraform repo. If the tag doesn't exist, customers will get a 404 when trying to download the module.

Customer Setup (one-time)

1. Get their **license ID** from Enterprise Portal (same one used for Docker image pulls)
2. Add credentials to `~/.terraformrc`:

```
None
credentials "proxy.replicated.com" {
  token = "<license-id>"
}
```

3. Reference the module in their Terraform config:

```
None
module "your-app" {
  source  = "proxy.replicated.com/<app-slug>/<repo-name>/github"
  version = "1.0.0"
}
```

Where `<app-slug>` is your app's slug, `<repo-name>` is the linked Terraform repo name, and `github` is the required provider suffix.

How It Works

When the customer runs `terraform init`, Terraform makes four sequential requests to `proxy.replicated.com` (a Cloudflare Worker):

1. **Discovery** — Worker returns a static registry manifest pointing to the modules API. No auth required.
2. **List versions** — Worker calls the Replicated API, which validates the license, checks the customer's channel, and returns the allowed `version_label` values. Only version numbers are returned to the customer.

3. **Download pointer** — Worker calls the Replicated API to create a short-lived server-side session backed by a JWT. The response includes an `X-Terraform-Get` header with the archive URL containing the JWT as a query parameter. The JWT is an opaque, short-lived token — the GitHub credential is stored server-side and never exposed to the customer.
4. **Tarball download** — Worker validates the JWT, retrieves the GitHub installation credentials from the server-side session, regenerates a fresh GitHub token, fetches the tarball from GitHub, and streams it back. GitHub redirects to a temporary `codeload.github.com` URL; the worker follows this redirect internally so the customer never sees it.

The customer **never sees** the GitHub token, repo URL, or any internal infrastructure. They only ever send their license ID and only ever talk to `proxy.replicated.com`.

Version Gating

Versions are gated by the existing release/channel system:

- Vendor promotes releases to channels with `version_label` values
- Customer's license is assigned to a channel
- The proxy only returns versions from releases on the customer's channel
- Licenses pinned to a specific channel sequence see only versions up to that sequence

Current Limitations (v1)

- No `terraform login` support — customers manually configure `~/.terraformrc`
- No tarball caching — all requests go through to GitHub
- Release version labels must match git tags in the Terraform repo with a `v` prefix (`1.0.0` → tag `v1.0.0`)
- Only `github` is supported as the provider suffix

Using Enterprise Portal

Search

- Enterprise Portal includes built-in full-text search. Customers can click the search button in the header or press **Cmd+K** (Mac) / **Ctrl+K** (Windows) to search across all pages in the current version. Results are filtered and highlighted as they type.
- No configuration is required. Search indexes all pages defined in `toc.yaml` and respects visibility rules.

Troubleshooting

Current Limitations

- Installation and upgrade instructions are available only for Embedded Cluster and Helm CLI installations. The Enterprise Portal does not provide instructions for KOTS or kURL.
- Air gap instance records do not appear until the customer creates one by uploading a support bundle or manually entering instance information.
- If you have many version branches and need to make a change across all versions, you must update each branch individually.

Changes Not Appearing

GitHub integration automatically syncs on push. If changes aren't appearing:

- Check the sync status in the Content tab
- Trigger manual sync from the Content tab (click "Sync")

API manual sync (see Connect Your Git Repo for how to get your `REPO_ID`):

```
None
replicated api post
/v3/app/<APP_ID>/enterprise-portal/content-repos/<REPO_ID>/sync
```

Pages Not Showing in Navigation

- Check `toc.yaml` includes the page
- Ensure file path matches `toc.yaml` reference
- Verify visibility rules (entitlements, channels)
- Check frontmatter `visible_when` conditions
- Confirm customer has required entitlements enabled

Version Branch Not Working

- Check branch exists: `git branch -r`
- Any branch other than `main` becomes a version
- For version gating to work, branch names must match release `version_label` values (see Naming Your Branches)

Helm Chart Reference Docs Not Generating

- Confirm the `name` in `toc.yaml` matches a chart name in the promoted release (not the release name — the Helm chart name from `Chart.yaml`)
- Reference docs generate asynchronously after promotion — allow a few minutes
- If the chart hasn't changed since the last promotion (same `values.yaml` and `Chart.yaml`), regeneration is skipped
- Values annotated with `# @docs:skip` or `# @docs:internal` are intentionally excluded

Terraform Module Docs Not Loading

- Verify the Terraform module repo is accessible by your GitHub App installation
- Check the `ref` (tag/branch) exists in the Terraform repo
- Confirm the module path is correct (subdirectory after `//`)
- Allow a few minutes for AI-generated content to process

Terraform Proxy Registry Not Working

- Verify the GitHub App is installed and has access to the Terraform repo (**Enterprise Portal > Content > Terraform Repository**)
- Confirm the Terraform repo is linked (this is a separate repo from the content repo)
- Check the customer's license is active, not expired or archived

- Ensure `version_label` on the channel release matches the git tag (with `v` prefix: version `1.0.0` requires tag `v1.0.0`)
- Confirm `~/.terraformrc` has credentials for `proxy.replicated.com`
- Only `github` is supported as the provider suffix in the module source